

Запускать так: ./run.sh

Жадник

Глобально хочется написать следующий алгоритм: сопоставим людям магазины в которые они должны идти, после чего будем уменьшать целевую функцию меняя целевой магазин для части людей на другой. Для этой идеи зададим такое условие:

Опустим факт того, что магазины надо открывать, а дадим возможность людям выбрать в какой магазин они хотят пойти и откроем все те, которые нужно хотя бы одному человеку, то есть сформулируем следующее:

$$\min_{(x_1, x_2, \dots, x_M) \in \{1, 2, \dots, N\}^M} \sum_{i=1}^M \text{dist}(\text{guy}_i, \text{store}_{x_i}) + \sum_{j=1}^N \text{cost}_j \cdot \text{is_open}_j,$$

где выполнено $\sum_{i: x_i=j} \text{need}_i \leq \text{capacity}_j$ и $\text{is_open}_j = I(\exists i : x_i = j)$.

Это можно переписать в линейную программу, которой мы однако пользоваться не будем. Для этого заменим $x_1, x_2, \dots, x_M$ на индикаторы того, что i -ый человек выбрал магазин j . Получим следующее:

$$\min_{\substack{\{x_{i,j}\}_{i=1, \dots, M; j=1, \dots, N} : x_{i,j} \in \{0,1\} \\ \{\text{is_open}_j\}_{j=1 \dots N} : \text{is_open}_j \in \{0,1\}}} \sum_{i,j} x_{i,j} \text{dist}(\text{guy}_i, \text{store}_j) + \sum_j \text{cost}_j \cdot \text{is_open}_j,$$

где выполнено $\forall i = 1, \dots, M \sum_{j=1}^N x_{i,j} = 1$; $\forall j = 1, \dots, N \sum_{i=1}^M \text{need}_i \cdot x_{i,j} \leq \text{capacity}_j \cdot \text{is_open}_j$.

Сперва напишем эвристику для ответа. Для всех людей отсортируем магазины в порядке увеличения длины до них. Рассмотрим некоторую перестановку людей. В порядке этой перестановки назначим человеку ближайший магазин, куда он может ещё пойти, то есть ближайший, остаточная вместимость которого больше или равна потребности человека.

На удивление эта эвристика уже сумела пройти первый тест на одной случайной перестановке. Метрики составили

3276471.40, 178669106.40, 5453881.94, 38438142.55, 10849389.49, 11969284.22

и это дало 3 балла как 3, 0, 0, 0, 0, 0.

Количество итераций у генерации перестановки было повышено до 10000, это не дало результатов, метрики составили

3276471.40, 178669106.40, 5413828.12, 38371121.06, 10803075.22, 11896122.65.

По этой причине количество итераций было уменьшено до 1, чтобы получить хоть какое-нибудь жадное решение.

Далее были написан 1-opt, который делал следующее:

Посмотрим на произвольного человека и заставим его пойти в другой магазин, а именно во все магазины по очереди и взглянем в каком случае метрика станет наименьшей. Если это не тот магазин, в который он ходит сейчас, то примем это изменение. Так будем делать, пока есть к кому применять изменения. До сходимости на каждом из тестов было проделано соответственно 1, 14, 26, 29, 220, 482 итерации. Метрики оказались следующими:

3269821.32, 154082909.65, 5167171.45, 37165227.47, 9552597.41, 8639293.94.

Это соответствует баллам 5, 0, 0, 0, 3, 3.

Далее был добавлен 2-орт:

Рассмотрим две двух людей которые ходят в разные магазины, возможно они хотели бы ходить наоборот, но 1-орт это не сумел, поскольку ни один из магазинов не может принять их обоих одновременно. Попробуем поменять их местами, если это уменьшит метрику, то примем такое изменение. Снова отработаем до сходимости.

Эта поправка не прошла больше тестов, зато улучшила метрики. Количество итераций до сходимости по тестам составило 0, 0, 55, 111, 78, 74, метрики составили

3269821.32, 154082909.65, 5137255.90, 37061561.87, 9468770.90, 8586540.59.

Визуализировав нынешнее решение наблюдается проблема, что несколько человек требовали открыть им какой-то магазин и стоимость открытия полупустых магазинов гораздо выше, чем трудность пройти в другой магазин. Более того, среди открытых магазинов походы людей скорее походят на кластеризацию по ближайшему магазину, что в целом похоже на истинно верное разбиение. Попытаемся исправить проблему открытия лишних магазинов: по очереди попробуем закрыть каждый магазин и отправить всех его посетителей в другие магазины старым способом. Если метрика станет хуже или нам это не удастся, то вернём как было. Это улучшило метрики, они составили

3269821.32, 31177877.16, 4792800.70, 28969918.34, 9153974.36, 7933857.27.

Что дало 17 баллов 5, 0, 3, 3, 3, 3. Также было попробованно пытаться закрыть раньше те магазины, в которых больше свободного места, то есть те, которые не понятно зачем открывать, потому что их потенциал далеко не раскрыт. Это улучшило метрики:

3269821.32, 23960419.05, 4801888.85, 28628889.58, 9195056.99, 7937979.43.

20 баллов: 5, 3, 3, 3, 3, 3.

Причём решение всё также похоже на то, что человек идёт в ближайший из открытых магазинов. Так что возможно достаточно просто открыть магазины правильно, а люди имеющимися операциями перейти в магазин получше или поменяться двум людям сумеют сами пойти туда, куда лучше. Если это свойство решение начнёт нарушаться тогда и подумаем про его реализацию.

Генетика

В постановке на конец первого подхода задача буквально звучит как назвать правильную битовую строку из 0 и 1 как именно надо открывать магазины. Такая постановка идеально подходит для генетических алгоритмов, поскольку именно по этой битовой строке можно и писать генетику.

Сперва напишем генератор популяции, для этого надо найти много валидных решений. В валидном решении может быть довольно разное количество объектов, а потому примерное количество открытых магазинов возьмём как нынешний `best_ans`. А именно с настраиваемой вероятностью изменим каждый конкретный бит его строки взятия магазинов. На четвёртом тесте возникла проблема, что популяция очень долго не генерировалась, из-за того, что `OneTwoOpt` применялся слишком долго. Из-за этого было принято решение, что если популяция генерируется дольше 15 секунд, то эта эвристика будет отключаться.

Напишем скрещивание. Делать будем банально: возьмём общие единички двух родителей, а из необщих выберем половину случайно. Если получилось невалидное решение то добавим ещё несколько случайно. Также было решено добавить лучшее решение в популяцию.

Отбор будет в формате соревнования на выбывание. Сколько-то лучших особей возьмём сразу. Среди оставшихся, пока их слишком много будем брать произвольную пару и того, у которого метрику хуже — убивать.

Вырожденной будем считать популяцию, если среднее хэминговых расстояний упало ниже какого-то процента от всей длины.

Настроим параметры, в данный момент есть параметры: размер популяции, количество скрещиваний, количество автоматически проходящих в следующий тур, вероятность начального шума при генерации и процент для вырожденности. Сперва выберем размер популяции:

Размер популяции	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
6	3269821.32	23294909.40	4794642.39	28551967.99	9178228.06	7928423.47
12	3269821.32	23196013.73	4796617.85	28619017.75	9205347.88	7926551.06
20	3269821.32	23153940.28	4799456.82	28556793.12	9183020.72	7952878.71
40	3269821.32	23108194.36	4794446.11	TL	9166215.09	7941635.27
60	3269821.32	23091644.68	4797463.14	TL	9203246.91	7942354.57

В целом метрики особо не меняются, только метрика второго теста немного улучшается. При этом время растёт, возьмём что-то небольшое, например 8.

Теперь выберем насколько случайно генерировать популяцию (сейчас в какой-то степени это генетика с миметикой), так как локальный поиск может менять маску, без этого алгоритм сходится куда медленнее).

p изм. бита	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.10	3269821.32	23857643.76	4795296.52	28696482.03	9199272.88	7928665.98
0.30	3269821.32	23376261.13	4796120.24	28540332.69	9168195.75	7950829.37
0.50	3269821.32	23432724.85	4794196.76	28606492.10	9175103.50	7910950.80
0.75	3269821.32	23248569.03	4787704.73	28523441.07	9145309.49	7910119.39
1	3269821.32	23629279.19	4800240.89	28576818.73	9175638.48	7927473.66

Лучшие метрики вышли при 75 процентах.

Теперь выберем число детей.

число детей	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
2	3269821.32	23300451.09	4795831.31	28560029.07	9173743.15	7950573.86
4	3269821.32	23966514.26	4792624.10	28555236.93	9176602.43	7939502.74
6	3269821.32	23478215.22	4792443.38	28640927.67	9184223.48	7952821.64
8	3269821.32	23080182.74	4803200.28	28554890.42	9194702.85	7925975.45
12	3269821.32	23236797.36	4790673.84	28621952.35	9170924.55	7941166.29
20	3269821.32	23451110.75	4796777.04	TL	9173469.00	7920553.31

Оптимальное 8. Остаётся выбрать сколько лучший сразу проходят без турнира:

Метрики от неё практически не изменяются и скорее всего это погрешность от вероятностных шагов. Поставим 0, чтобы убрать непонятную механику.

Однако даже после настройки генетика не особо добивается успеха. Метрики не особо изменились по сравнению с предыдущим результатом.

Нынешнее решение уже насытило все магазины достаточно сильно и хотелось бы просто выбрать магазины лучше. Генетика же не может рассмотреть маленькие изменения, потому как работает с очень маленьким суммарным покрытием. Попробуем добавить какую-то мутацию, чтобы затрагивать как можно больше магазинов.

всегда выживают	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0	3269821.32	23170732.92	4799950.65	28635275.02	9184561.08	7948385.01
2	3269821.32	23365566.26	4798942.40	28700063.19	9157640.68	7938723.78
4	3269821.32	23499924.74	4795235.87	28591610.52	9163283.58	7936070.32
6	3269821.32	23263392.61	4798446.69	28579868.73	9147024.72	7951352.45

Попробуем самую простую мутацию для наших целей: добавим несколько магазинов в маску и пересчитаем метрику. При добавлении случайных битов метрики составили:

3269821.32, 23081307.85, 4792545.34, 28570234.24, 9161673.13, 7929047.19.

То есть всё так же поменялись на шум.

Видимо имеющееся решение в целом близко к генетическому оптимуму и магазины стоит выбирать тоньше. Глобально решение правдоподобное и метрики надо улучшать маленькими аккуратными улучшениями.

Tabu search